

هندسة البرمجيات (Software Engineering)

الفصل السادس: الإستراتيجية التطويرية (Development Strategy)

المخطط العام – Outline

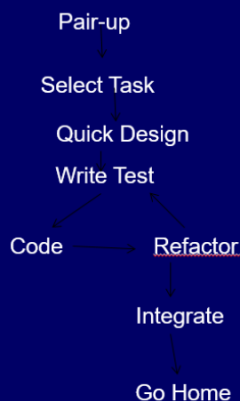
- التعريف: Definition
- البرمجة الثنائية: Pair programming
- الملكية الجماعية: Collective ownership
- معيار الشيفرة: Code standard
- أسبوع عمل من 40 ساعة: 40 hours week
- إعادة هيكلة الشيفرة: Refactoring

التعريف (Definition)

- ال Code (الشفرة البرمجية) هي جوهر أي برنامج. كلما كانت الشفرة أفضل، كانت الاستجابة للمدخلات أفضل.
- تعتمد (Extreme Programming - XP) على الترميز كالنشاط الأساسي في عملية التطوير.
- يعتبر المبرمج هو محور منهجية XP.
- في XP، إن لم تكتب Code في نهاية اليوم، فكأنك لم تفعل شيئًا.

XP Process

An XP developer's daily routine



البرمجة الثنائية (Pair Programming)

Pair-up:

- يعمل المطورون في أزواج، ويتشاركون محطة العمل. إذا كنت لا تزال تعمل على مهمة، فقد يكون زوجك معروفًا مسبقًا، وإلا ستحتاج للعثور على مطور آخر من الفريق لتشكيل ثنائي.

Select Task:

- إذا لم تكن لا تزال تعمل على مهمة حالية، يجب على الثنائي اختيار Engineering Task جديدة للعمل عليها.

Quick Design:

- التصميم في هذه المرحلة يعني "كيف سنحل هذه المهمة؟" - أي وضع تصور مبدئي للحل.

Write Test:

- ابدأ بكتابة Test Case وتأكد أن هذا الاختبار يفشل قبل البدء بالترميز. قم بتوسيع الاختبار ليغطي كل ما يمكن أن يتسبب في خلل. واطلب التوضيح من فريق العملاء عند الحاجة.

Code:

- اكتب الكود المطلوب لإتمام المهمة، مع مراعاة كتابة القدر الكافي فقط من Code لتمرير Test Case.

Refactoring:

- راجع الشفرة وابحث عن مناطق يمكن تبسيطها وتحسينها.
- توقف عن Refactoring عندما تصل إلى مستوى مناسب من البساطة.

Integrate:

- أضف المكون البرمجي المكتمل إلى نظام Integration/Build Machine.
- تحقق من أن Class المدمجة تجتاز جميع Tests.
- إذا استمرت مشاكل الدمج، قم بإلغاء الشيفرة وابدأ من جديد في اليوم التالي.

Go Home:

- اتبع ممارسة XP الخاصة بالعمل 40 ساعة أسبوعيًا.
- غادر في الساعة 5:00 مساءً.

البرمجة الثنائية – Pair Programming

المفهوم الأساسي:

- يعمل مبرمجان معًا على نفس التصميم أو الخوارزمية أو الشيفرة البرمجية أو اختبار (Test Case).
- ال Pairing عملية ديناميكية (ليست ثابتة).
- تعزز التواصل والإنتاجية وتحسن جودة الشيفرة.
- مفيدة لتدريب الموظفين الجدد، وكذلك لتبادل الخبرات بين الموظفين الحاليين (Cross-Training).

القواعد – Rules:

- جميع جلسات البرمجة تتم جنبًا إلى جنب: شخصان على شاشة واحدة.
- يجب أن يفهم كلا الطرفين كل سطر من Code، ويوافقا عليه.
- يتم تبديل الأدوار (الكاتب والمراقب) من حين لآخر لضمان التفاعل المشترك.

نتائج الأبحاث – Research Findings

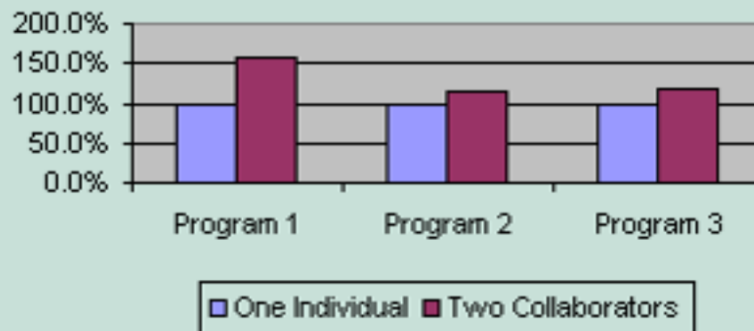
تُظهر أبحاث Pair Programming أن:

- الثنائيات لا تستهلك عدد ساعات عمل أكثر من الأفراد.
- الثنائيات تنتج أخطاء أقل.
- الثنائيات تكتب أسطر كود أقل.
- الثنائيات تستمتع بالعمل أكثر.

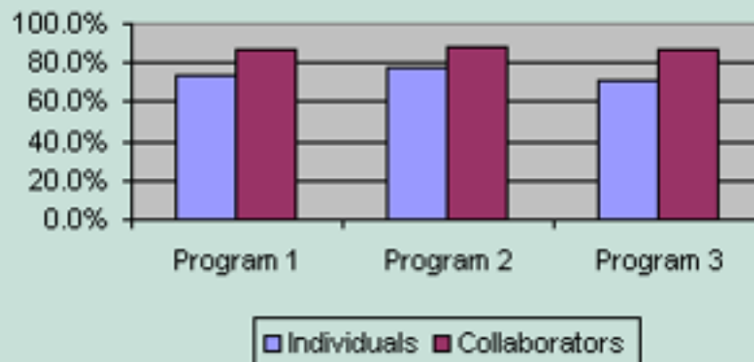
تجربة جامعة يوتا – University of Utah Experiment

- أنفقت الثنائيات وقتًا أكثر بنسبة 15% على البرمجة مقارنةً بالأفراد.
- الكود المكتوب من قبل الثنائيات نجح في اختبارات أكثر.
- الثنائيات أنتجت نفس الوظائف بأسطر أقل من الكود.
- البرمجة في بيئة تحقق نتائج ملموسة بسرعة ممتعة وتساعد على التعلم بشكل أسرع.

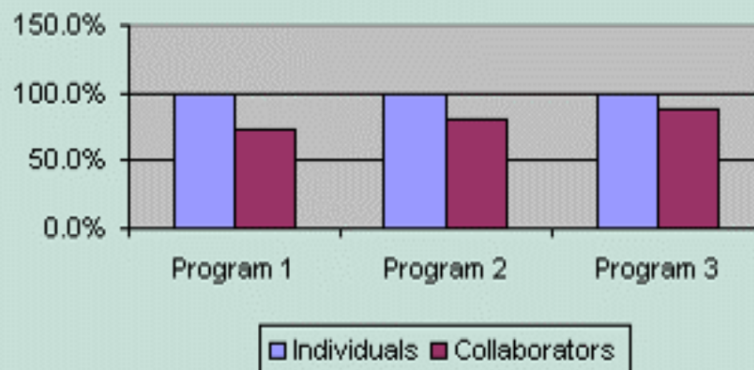
Relative Time: One Individual vs Two Collaborators



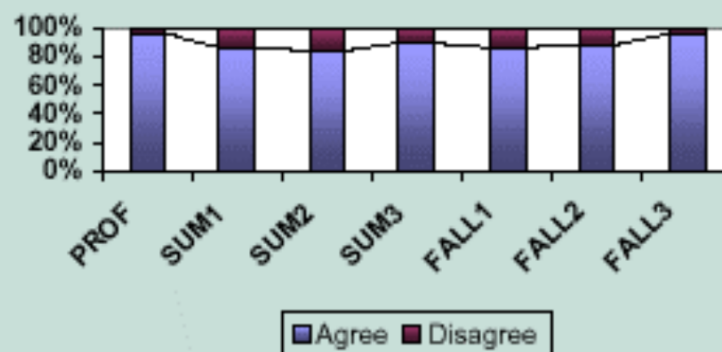
Post Development Test Cases Passed



Lines of Code



ENJOY the work more because of Pair Programming



استنتاجات تجربة جامعة يوتا – Conclusions

الفوائد - Benefits

- تُكتشف الأخطاء في أثناء الكتابة، وليس في مرحلة الاختبار.
- عدد الأخطاء النهائية أقل إحصائيًا.
- التصاميم الناتجة تكون أفضل.
- الشيفرة تكون أقصر.
- الفريق يحل المشاكل بشكل أسرع.

العيوب – Disadvantages

1 فرق المهارات – Skill Difference:

قد يتسبب الفرق الكبير في المهارات إلى سيطرة أحد المبرمجين، أو تحوّل الجلسة إلى درس تدريبي، مما يُضعف الفائدة.

2 عدم إنجاز العمل – Not Getting Work Done:

بعض الأشخاص يتحولون للجوانب الاجتماعية أثناء الجلسة، أو يشعرون بعدم الراحة عند وجود شخص يراقبهم، مما يضعف فعالية العمل.

3 غرور المطورين – Developer Egos:

قد تظهر صراعات بين المبرمجين الأكثر خبرة حول "الطريقة الأفضل"، وقد تتحول الجلسة إلى نزاع بدلاً من تعاون.

4 وقت التهيئة – Startup Time:

يحتاج الأمر لبعض الوقت ليتأقلم المطورون مع بيئة العمل الثنائية.

5 تقييم الأفراد – Evaluating Programmers:

من الصعب تقييم كل فرد حسب مساهمته بدقة، وهو أمر يمكن تجاوزه في البيئة الأكاديمية، لكنه يمثل تحديًا حقيقيًا في المجال الصناعي.

مفاجآت إيجابية – Positive Surprise

- كثير من الأشخاص جربوا Pair Programming متوقعين أنهم لن يحبوه، لكنهم أعجبوا به بشكل غير متوقع بعد التجربة.

الملكية الجماعية للكود

(Collective Ownership)

? لماذا نحتاجها؟

- ماذا يحدث عندما يغيب شخص أساسي بسبب الإجازة، أو ظروف طارئة، أو ينتقل لوظيفة جديدة أو يتقاعد فجأة؟
- كم من الوقت ستقضيه لتدريب بديل؟
- ال **Collective Code Ownership** هي الحل: توزيع المسؤولية على جميع المطورين.

الفكرة الأساسية:

- ال **Collective Ownership** تعني أن الجميع مسؤول عن جودة الكود.
- لا يوجد شخص يمتلك جزءاً معيناً من النظام.
- أي شخص يمكنه تعديل أي جزء في **codebase** في أي وقت.
- المسؤولية موزعة، وليست مركزة على أفراد.

السمات الرئيسية:

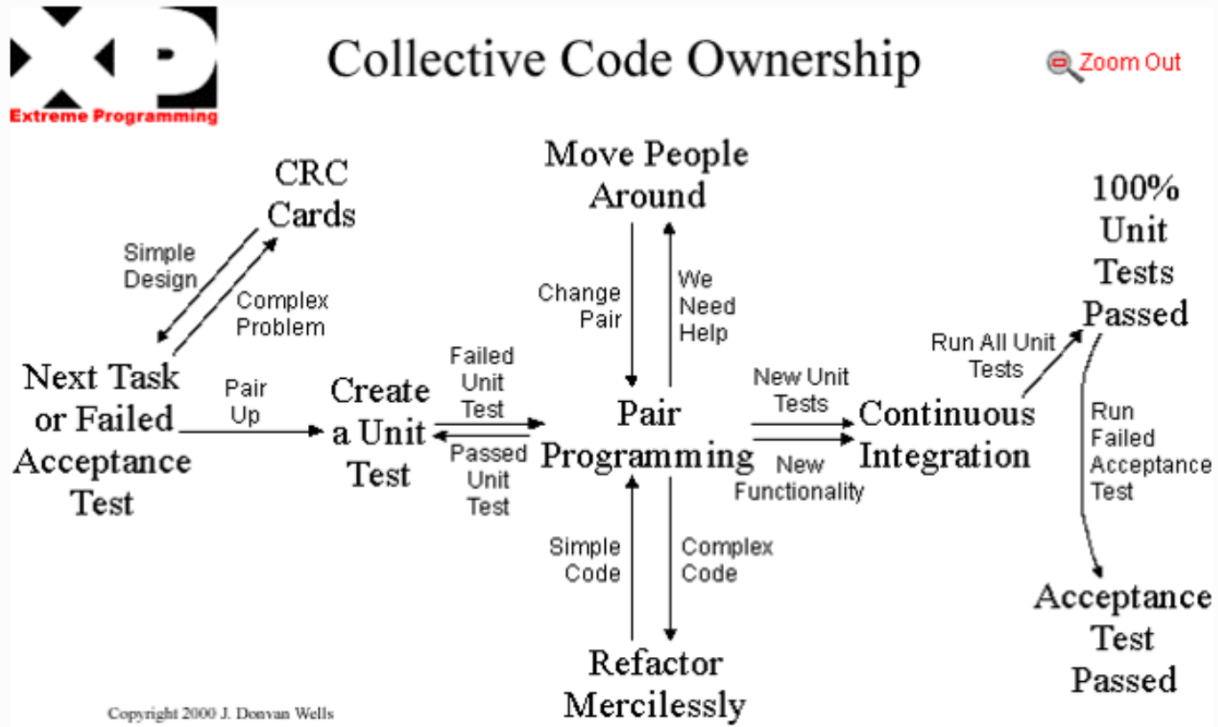
1. أي عضو في الفريق يمكنه إضافة أو تعديل الكود في أي وقت.
2. كل فرد مسؤول عن النظام بالكامل.
3. تشجع على البساطة: تمنع إدخال الكود المعقد للنظام.
4. تتيح اكتشاف وإعادة هيكلة الكود بسرعة.
5. تزيد من المسؤولية الفردية والتمكين الشخصي.
6. تقلل من مخاطر المشروع: تنشر المعرفة بالنظام على كل الفريق.

كيف نطبق الملكية الجماعية؟

لجعل **Collective Code Ownership** تعمل بفعالية، يجب أن نتخلى عن بعض الأنانية البرمجية:

- ✨ افخر بكود الفريق، لا بكودك الشخصي.
- ✨ لا تشتك إن قام شخص بتحسين كودك – استمتع بتطوره حتى وأنت بعيد.
- ✨ ناقش الرؤية التصميمية مع الفريق بدلاً من فرض رأيك الشخصي.

الترميز في منهجية البرمجة القصوى (XP Project Coding)



الفوائد – Benefits ✓

- فرق تطوير البرمجيات غير مستقرة بطبيعتها: الناس يرحلون وينضمون باستمرار، لذا لا يمكن الاعتماد على شخص واحد طويل الأمد.
- المعرفة المشتركة بالكود:
- يساعد المطورين على فهم أكبر لهيكل النظام.
- يتيح الاستفادة من خبرات بعضهم البعض.
- كود أبسط: العيون المتعددة تكتشف التعقيد بسرعة.
- إنجاز أسرع: من يحتاج لتعديل، يعدل فورًا دون انتظار.
- تحسين جودة الكود: الجميع يصلح المشاكل عند رؤيتها: التكرار، الأسماء الغامضة، التصميم السيئ... كل شيء قابل للتحسين من أي شخص.

السلبات – Drawbacks ✗

- شخص واحد قد يتحول إلى "مُصلح الأخطاء" الدائم: وهذا دور مرهق وغير محبوب، لذلك من المهم توزيع مسؤولية تصحيح الأخطاء بالتساوي.
- زيادة عدد المتدخلين تعني احتمالية تضارب في الكود أعلى.
- بعض المطورين قد لا يتقبلون التعديل على كودهم الخاص: خصوصًا من أنشأوا وحدة معينة، قد يشعرون بأنها "ملكهم"، ويرفضون تدخل الآخرين.

معايير كتابة الكود (Coding Standards)

ما هي معايير الكود؟

- وثيقة **Coding Standards** تُحدد للمطورين كيفية كتابة الكود.
- بدلاً من أن يستخدم كل مبرمج أسلوبه الخاص، يتبع الجميع نفس الأسلوب المحدد في الوثيقة.
- النتيجة: مشروع كبير مكتوب بأسلوب موحد - دون تفاوت في الأنماط بين أجزاء البرنامج.

فوائد المعايير:

- تسهّل فهم الكود.
- تجعل أي مطور يمكنه قراءة الكود يعرف ما يتوقعه في أي جزء من النظام.
- تعزز قابلية الصيانة والعمل الجماعي.

أهمية المعايير في XP:

- في **Extreme Programming (XP)**، تعمل كمطور ثنائي (pair).
- وجود معايير متفق عليها يزيل الخلافات التافهة حول:
- تنسيقات الكود (**Formatting**)، تسمية المتغيرات (**Naming**)
- بدون معايير، يصبح تبديل الثنائيات صعباً لأن كل مطور له أسلوبه المختلف.

مبادئ المعايير الجيدة يجب أن:

- تطلب أقل جهد ممكن، مع احترام قاعدة **Once and Only Once** (لا تكرار للكود).
- تركز على التواصل الواضح من خلال الكود.
- يتم تبنيها طوعاً من الفريق، لا فرضها.
- يجب أن تُصمم المعايير مع أهداف الصيانة في الاعتبار.
- المعيار الجيد يجعل أي مطور يستطيع العمل على أي كود طالما التزم بالمعايير.

شكل معايير الكود قد تكون:

- مجموعة تعليمات بسيطة.
- أو مستندات تفصيلية مع أمثلة وقواعد واضحة.

مثال على معيار كامل (لغة Java)

 Oracle Java Coding Conventions (Official)

أسبوع العمل 40 ساعة (Hour Week-40)

المفهوم العام

في XP، يتميز التطوير بـ:

- التواصل المكثف.
- السرعة العالية.
- يعمل المطورون في بيئة يشكل التغيير المستمر مصدر ضغط دائم.

السؤال الأهم: كيف نحافظ على الجودة مع ساعات

عمل إضافية مرهقة؟

- الجواب ببساطة: لا يمكن ذلك.
- أحد المبادئ الأساسية في XP هو:
- "لا يتم أبداً التنازل عن جودة العمل."

فلسفة 40 ساعة

- لو تم توجيه طاقة المطور بالكامل نحو المشروع لمدة 8 ساعات يوميًا (دون انشغالات خارجية)، فإن أسبوع العمل 40 ساعة سيكون كافيًا لتحقيق نتائج مميزة.

الفكرة:

- ✓ العمل بعدد ساعات يمكن للفريق الاستمرار بها دون تدهور الأداء.
- ✓ هذا العدد غالبًا ما يكون 40 ساعة أسبوعيًا أو قريبًا من ذلك.

ما المشكلة في ساعات العمل الزائدة؟

- انخفاض جودة الكود.
- استنزاف طاقة الفريق.
- ضغوط نفسية ومهنية.
- تخلق ما يسمى بـ "السوق السوداء للوقت":
- مثال: العقود تحدد 8 ساعات يوميًا، لكن الإدارة تضغط على الفريق للعمل أكثر بدون مقابل إضافي.
- الموظفون برواتب ثابتة يخسرون وقتهم الشخصي دون أي فائدة.

قاعدة XP الصارمة ⚠️

القاعدة بسيطة: 📖

- "لا يمكنك العمل أسبوعين متتاليين من العمل الإضافي".
- أسبوع واحد من الضغط والعمل لساعات أطول؟ ممكن.
- لكن إذا جئت يوم الاثنين وقلت: "علينا العمل لساعات إضافية مرة أخرى هذا الأسبوع"،
- 🔥 فهذا يعني أن لديك مشكلة في التخطيط أو في إدارة المشروع – وليست مشكلة يمكن حلها بالمزيد من الساعات.

إعادة الهيكلة البرمجية (Refactoring)

? ما هو Refactoring?

- ال Refactoring هو تحسين الكود دون تغيير وظيفته.
- عملية مستمرة تهدف إلى تبسيط الكود والتصميم والاختبارات وحتى منهجية XP نفسها.

مثال بسيط (Simple Example): 🧱

قبل: 🏷️

```
public class Person {  
    private String firstName;  
    public void setFirst(String n) {  
        this.firstName = n;  
    }  
}
```

بعد تحسين اسم المتغير: 🏷️

```
public class Person {  
    private String firstName;  
    public void setFirst(String firstName) {  
        this.firstName = firstName;  
    }  
}
```



```
public class Person {  
    private String firstName;  
    public void setFirstName(String firstName) {  
        this.firstName = firstName;  
    }  
}
```

لماذا نقوم بـ Refactoring؟

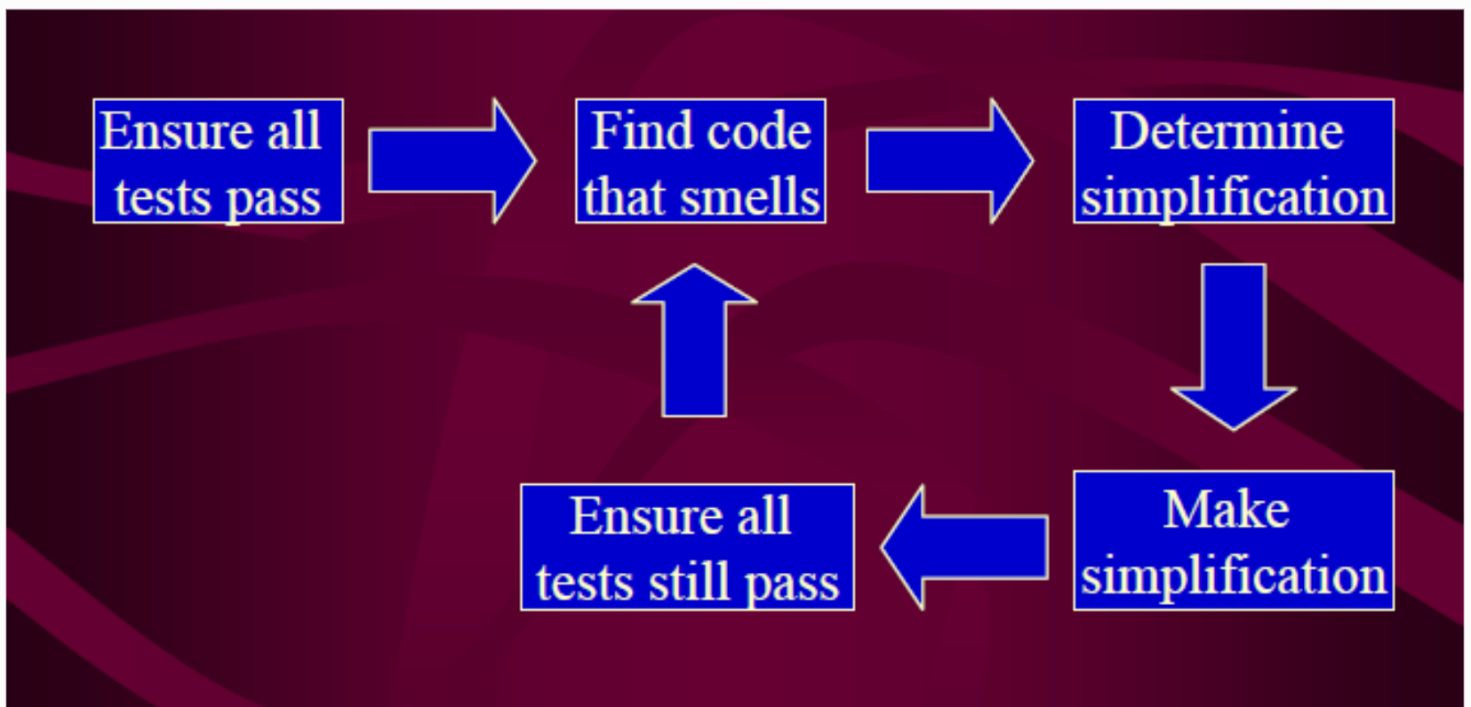
- **تحسين التصميم:** بدون إعادة هيكلة، تصميم النظام يتدهور مع الوقت.
- **سهولة الفهم:** من سيعدّل الكود لاحقًا هو الأهم، وليس المترجم.
- **اكتشاف الأخطاء:** أثناء Refactoring، يتم تحليل الكود بعمق، مما يفضي إلى كشف الأخطاء.
- **تسريع التطوير:** تصميم جيد يسرّع عملية التطوير ويحافظ على استقرار الأداء.

خصائص Refactoring

1. تحويل من كود إلى كود (source-to-source).
2. يبقى داخل نفس اللغة (Java → Java).
3. لا يغيّر السلوك العام للبرنامج.
4. بدأ في البرمجة الكائنية (OOP) لكنه قابل للتطبيق في لغات غير كائنية.
5. يعتمد على اكتشاف ما يسمى بـ Code Smells (روائح الكود) – أي
6. إشارات تدل على سوء في الكود.

إشارات وجود مشكلة في الكود (Code Smells)

- وهي مؤشرات تدل على أن هناك مشكلة في الكود.
- مؤشرات على وجود خطأ أو مشكلة في الكود.
- يمكن أن تظهر في كود الإنتاج (Production Code) أو كود الاختبار (Test Code).



خطوات إعادة الهيكلة (Refactoring):

1- الكود المكرر - Duplicated Code

- إذا رأيت نفس البنية البرمجية مكررة في أكثر من مكان، فمن الأفضل توحيدها.
- أبسط حالات التكرار هي وجود نفس التعبير في دالتين داخل نفس الكلاس (Class).
- استخدم **Extract Method** لاستخرج الكود المشترك واستدع الدالة من كلا المكانين.
- حالة تكرار أخرى شائعة هي وجود نفس التعبير في Subclasses متشابهة.
- استخدم **Pull Up Method** لنقل الكود إلى الكلاس الأعلى (Superclass).

1 استخراج دالة – Extract Method

تُستخدم عندما يكون هناك جزء من الكود يمكن تجميعه معًا. يتم استخراج هذا الجزء وتحويله إلى دالة جديدة باسم يوضح وظيفتها.

قبل: 

```
void printOwing(double amount) {  
    printBanner();  
    // طباعة التفاصيل  
    System.out.println("name:" + _name);  
    System.out.println("amount" + amount);  
}
```

مثال آخر: 

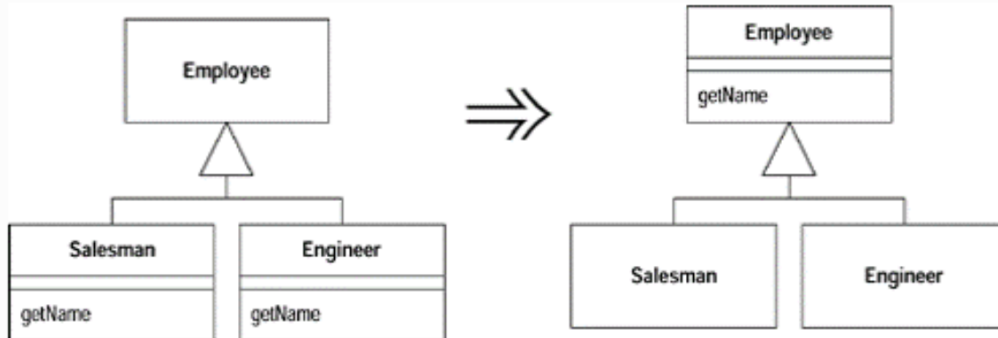
```
void printReceipt(double a) {  
    // طباعة التفاصيل  
    System.out.println("name:" + _n);  
    System.out.println("amount" + a);  
}
```

بعد إعادة الهيكلة: 

```
void printOwing(double amount) {  
    printBanner();  
    printDetails(amount); }  
void printReceipt(double a) {  
    printBanner();  
    printDetails(a); }  
void printDetails(double amount) {  
    System.out.println("name:" + _name);  
    System.out.println("amount" + amount);  
}
```

2 رفع الدالة – Pull Up Method

- إذا وُجدت دوال في Subclasses مختلفة تُنتج نفس النتائج، يمكن نقلها إلى Superclass.
- إزالة التكرار تجعل الكود أسهل في الصيانة والفهم.



```
1: public abstract class Vehicle
2: {
3:     public void Turn(Direction direction)
4:     {
5:         // code here
6:     }
7: }
8:
9: public class Car : Vehicle
10: {
11: }
12:
13: public class Motorcycle : Vehicle
14: {
15: }
16:
17: public enum Direction
18: {
19:     Left,
20:     Right
21: }
```

```
1: public abstract class Vehicle
2: {
3:     // other methods
4: }
5:
6: public class Car : Vehicle
7: {
8:     public void Turn(Direction direction)
9:     {
10:         // code here
11:     }
12: }
13:
14: public class Motorcycle : Vehicle
15: {
16: }
17:
18: public enum Direction
19: {
20:     Left,
21:     Right
22: }
```

2- دالة طويلة – Long Method

- كلما طالت الدالة أصبح فهمها أصعب.
- الحل في الغالب هو Extract Method.
- قاعدة عامة: إذا كانت الدالة أطول من شاشة واحدة فهي طويلة جدًا.

```
void printFormatted(string text) {
    System.out.println("Copyright (c) 1006, Siemens AG");
    System.out.println("Author: Michael Stal");
    printRest(text);
}
```



```
void printFormatted(string text) {
    printHeader();
    printRest(text);
}

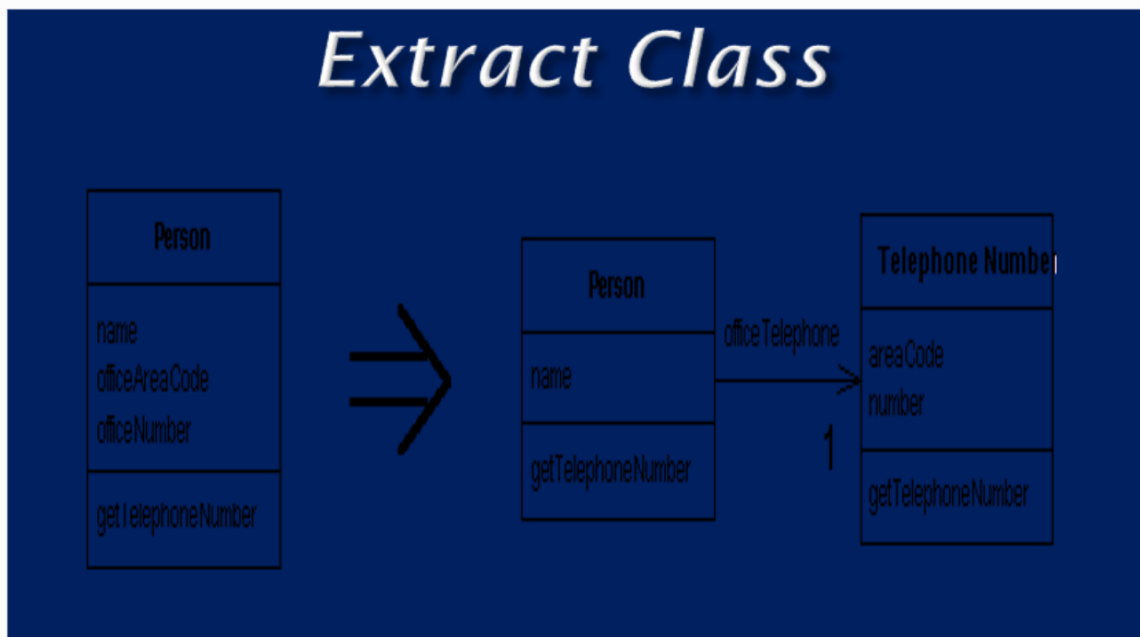
printHeader() {
    System.out.println("Copyright (c) 1006, Siemens AG");
    System.out.println("Author: Michael Stal");
}
```

3- كلاس ضخيم – Large Class

- عندما يحاول الكلاس (Class) تنفيذ الكثير من المهام، غالبًا يحتوي على عدد كبير من المتغيرات.
- هذا يؤدي إلى تكرار الكود.
- الحل: استخدم Extract Class أو Extract Subclass.

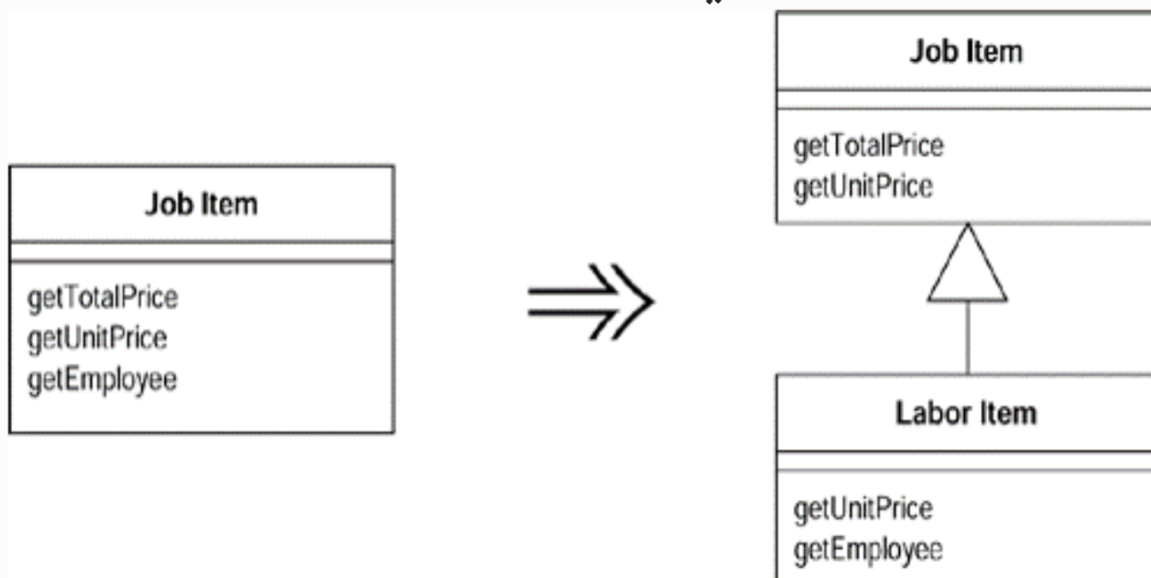
1 Extract Class

- إذا كان كلاس يؤدي وظيفتين، قم بإنشاء كلاس جديد وانقل الوظائف والخصائص المتعلقة إليه.



2 Extract Subclass

- عندما يحتوي الكلاس على سلوك مخصص لبعض الحالات فقط.
- أنشئ Subclass يحتوي على هذا السلوك الخاص.



4- قائمة معلمات طويلة – Long Parameter List

- القوائم الطويلة للمعلمات صعبة الفهم وتغييرها يتطلب تعديلات متكررة.
- استخدم **Replace Parameter with Method** عندما يمكنك جلب البيانات من كائن معروف.



Replace Parameter with Explicit Methods

كائن ما يستدعي دالة، ثم يُمرر ناتج تلك الدالة كمعامل لدالة أخرى. لكن الكائن المستقبل يمكنه أيضًا استدعاء نفس الدالة مباشرة للحصول على نفس القيمة.

قبل:

```
void setValue(String name, int value) {  
    if (name.equals("height")) {  
        _height = value;  
        return; }  
    if (name.equals("width")) {  
        _width = value;  
        return; } }
```

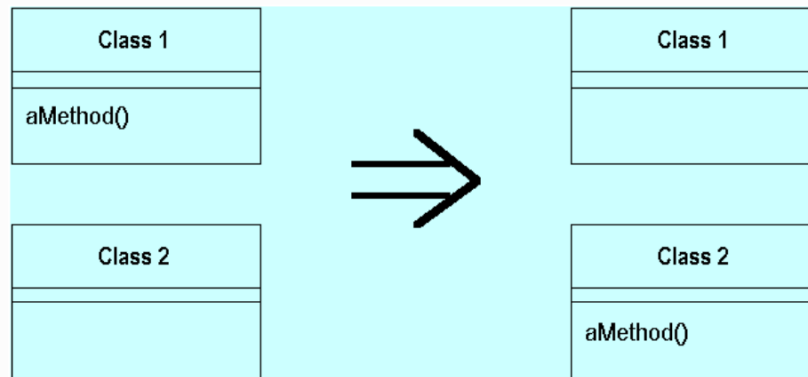
بعد:

```
void setHeight(int arg){  
    _height = arg; }  
void setWidth(int arg) {  
    _width = arg; }
```

5- الحسد الوظيفي – Feature Envy

- عندما تهتم دالة بـ كلاس آخر أكثر من كلاسها الحالي.
- الحل: استخدم **Move Method** لنقل الدالة إلى الكلاس الذي تستخدمه أكثر.

Move Method



قبل: 

```
class Project {
    Person[] participants;
}
class Person {
    int id;
    boolean participate(Project p) {
        for (int i = 0; i < p.participants.length; i++) {
            if (p.participants[i].id == id)
                return true;
        }
        return false;
    }
}
... if (x.participate(p)) ...
```

بعد: 

```
class Project {
    Person[] participants;
    boolean participate(Person x) {
        for (int i = 0; i < participants.length; i++) {
            if (participants[i].id == x.id)
                return true;
        }
        return false;
    }
}
class Person { int id; }
... if (p.participate(x)) ...
```

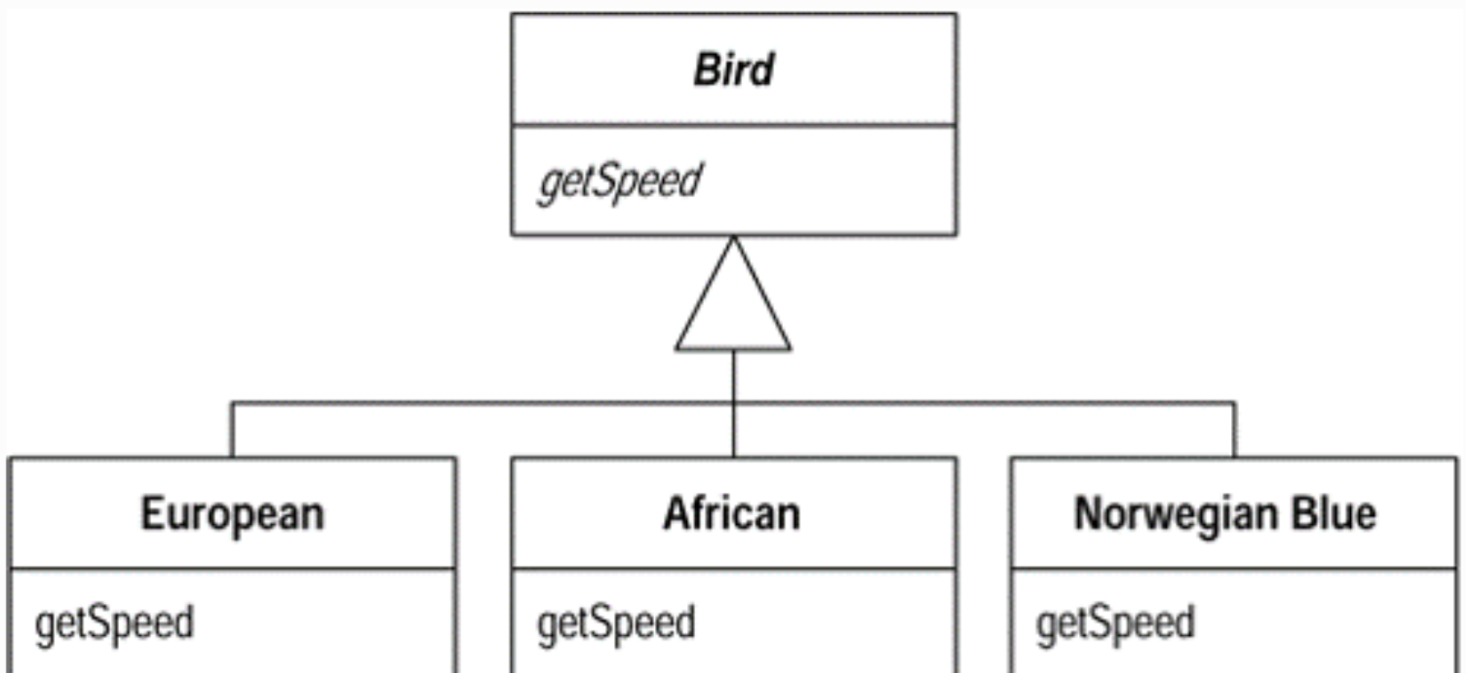
6- العبارات الشرطية – Switch Statements

- عندما ترى عبارة switch، فكر في استخدام Polymorphism (تعدد الأشكال).
- غالبًا يتم التبديل حسب نوع (type code).
- استخدم Extract Method ثم Move Method لنقلها إلى الكلاس المناسب.
- ثم استخدم Replace Conditional with Polymorphism.

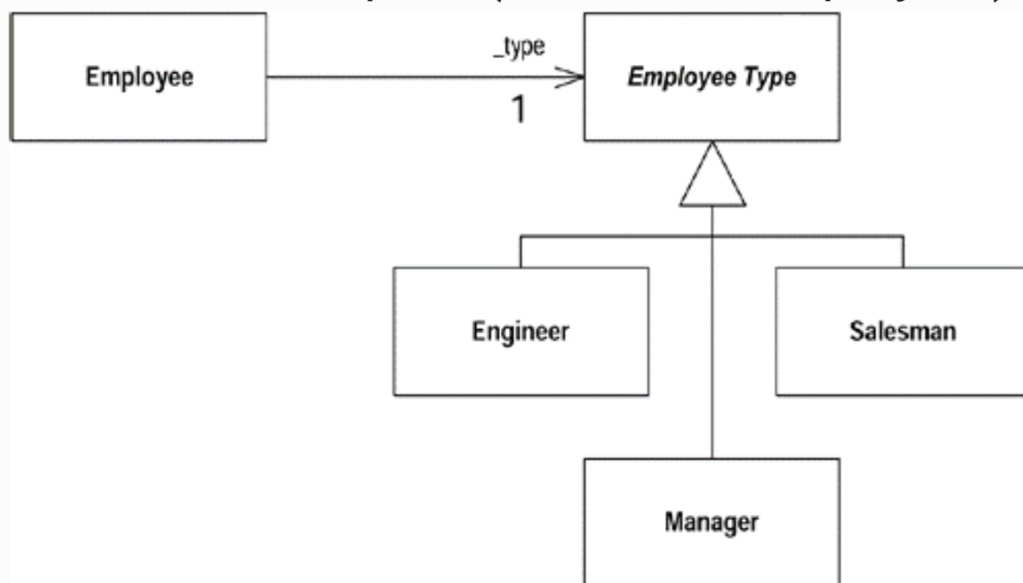
🌟 Replace Conditional with Polymorphism

مثال: 

```
Double getSpeed() {  
    switch (_type) {  
    case EUROPEAN:  
        return getBaseSpeed();  
    case AFRICAN:  
        return getBaseSpeed() - getLoadFactor() * _numCoconuts;  
    case NORWEGIAN_BLUE:  
        return (_isNailed) ? 0 : getBaseSpeed(_voltage);  
    { {
```



```
class EmployeeType {
int payAmount(Employee emp) {
switch (getTypeCode()) {
case ENGINEER:
return emp.getMonthlySalary();
case SALESMAN:
return emp.getMonthlySalary() + emp.getCommission();
case MANAGER:
return emp.getMonthlySalary() + emp.getBonus(); default:
throw new RuntimeException("Incorrect Employee"); } } }
```



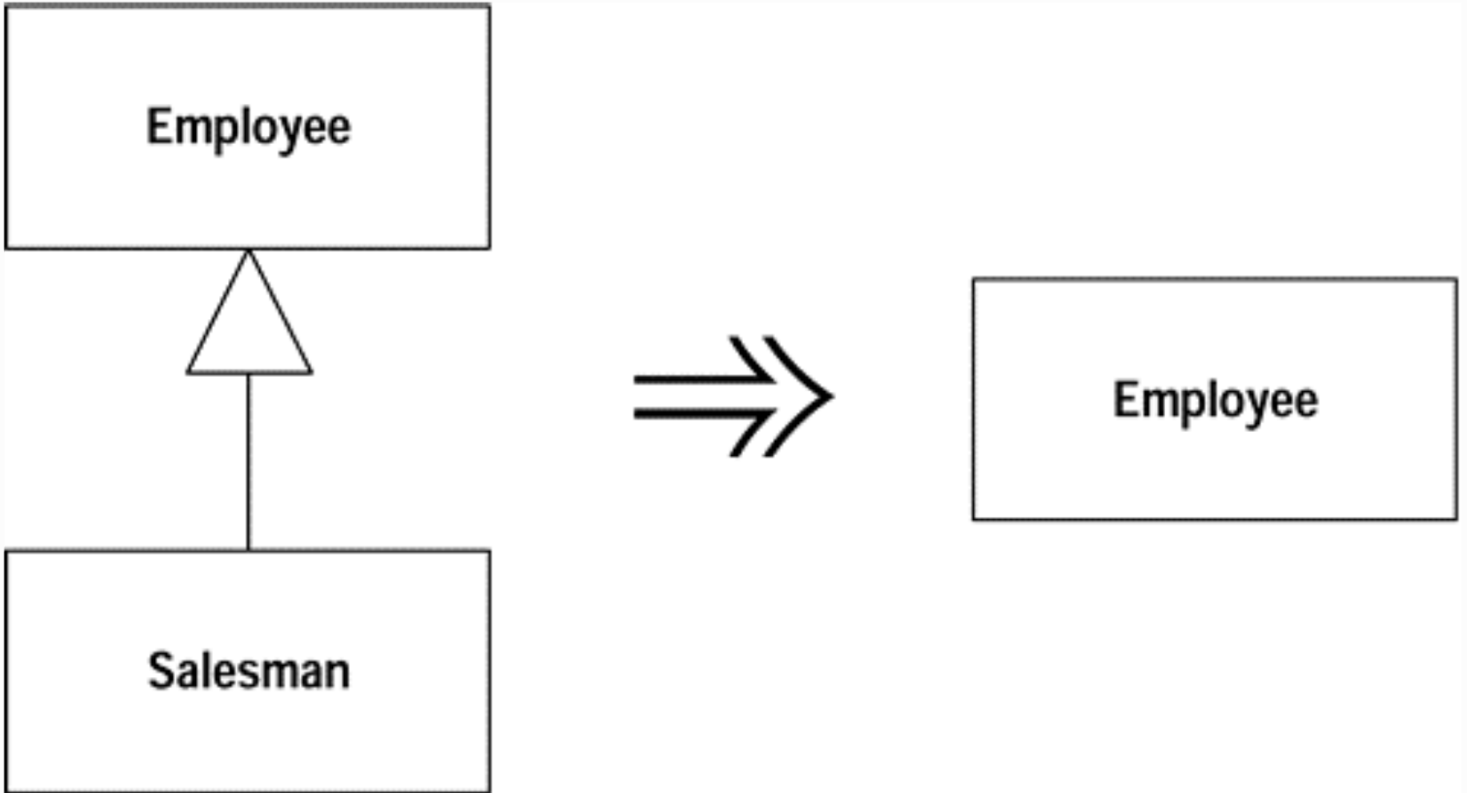
```
class Engineer {
int payAmount(Employee emp) {
return emp.getMonthlySalary(); } }
class Salesman {
int payAmount(Employee emp) {
return emp.getMonthlySalary() + emp.getCommission(); } }
class Manager { int payAmount(Employee emp) {
return emp.getMonthlySalary() + emp.getBonus(); } } abstract
class EmployeeType { abstract int payAmount(Employee emp); }
```

7- كلاس كسول – Lazy Class

- كل كلاس تُنشئه يكلف وقتًا ومالًا في الصيانة.
- إذا كان الكلاس لا يقوم بوظائف حقيقية، احذفه.
- استخدم **Collapse Hierarchy** لدمج كلاس فرعي مع الرئيسي إذا لم يكن هناك اختلاف جوهري.
- استخدم **Inline Class** إذا لم يعد الكلاس مفيدًا.

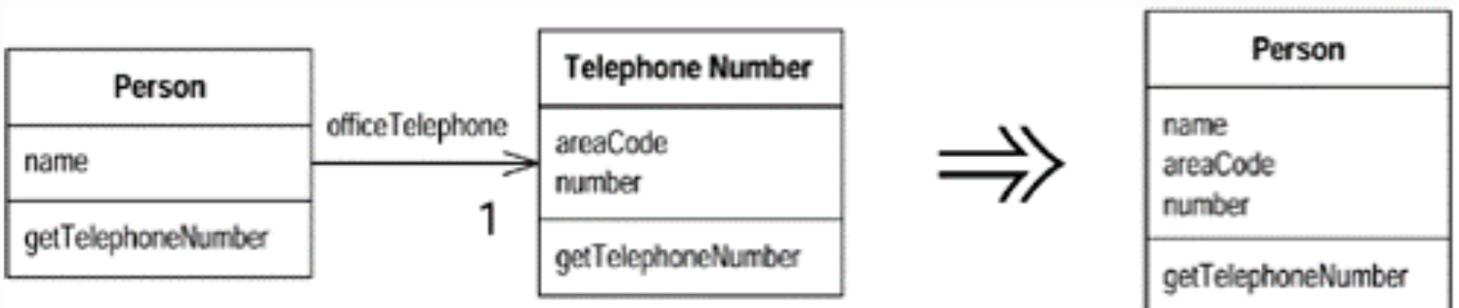
1 Collapse Hierarchy

- دمج Subclass و Superclass عندما لا يكون هناك فرق حقيقي.



2 Inline Class

- عكس Extract Class، يُستخدم عندما يصبح الكلاس غير مهم بعد إزالة مسؤولياته.



8- الكود الميت – Dead Code

- وجود كود غير مستخدم أو مُعلق (مُعلق = مكتوب داخل تعليقات) ليس جيدًا.
- لا تتركه في الكود "لعلّك تحتاجه لاحقًا".
- اختبر إن كنت بحاجة له الآن، وإذا لم تكن، احذفه فورًا.
- وجوده يُربك المطورين الآخرين ويضيع وقتهم.

9- الأرقام السحرية – Magic Numbers

- استبدل الأرقام السحرية بثوابت معنونة (Symbolic Constants).
- أنشئ ثابتًا واضح الاسم يعبر عن المعنى، واستبدل الرقم به.

قبل: 🔍

```
static double ConvertPoundsToKilos(  
double pounds) {  
return pounds / 2.2;  
}
```

بعد: 🔍

```
static final double KILO_CONVERSION_FACTOR = 2.2;  
static double ConvertPoundsToKilos(double pounds) {  
return pounds / KILO_CONVERSION_FACTOR;  
}
```